

# Miniproject: Simple Banking

**Aim:** To develop and deploy a decentralized **Simple Banking** application on the Ethereum Sepolia Testnet where:

1. Every new account registers and receives **100 tokens** from a total supply of **1,000,000 tokens**.
2. Users can **deposit** ETH, **withdraw** ETH, and **transfer** tokens.
3. Account balances are retrievable via `balanceOf(address user)`.
4. A Web3 frontend provides controls for Deposit, Withdraw, Transfer, Check Balance, and Listing all accounts.

## Steps:

### 1. Wallet & Testnet Setup

- Installed the **MetaMask** browser extension and configured browser settings to ensure MetaMask serves as the default Web3 provider.
- Connected MetaMask to the **Ethereum Sepolia Testnet** for zero-cost testing.
- Acquired free test funds (Sepolia ETH) via the **Google Web3 Faucet** to cover deployment and transaction gas fees.

### 2. Smart Contract Development & Compilation

- Authored the SimpleBank.sol contract in Solidity (^0.8.20) defining:
  - 1,000,000 initial token supply with a 100-token bonus per new registration.
  - Payable `deposit()`, secure `withdraw()`, internal `transfer()`, and `balanceOf()` getter.
- Opened **Remix IDE** (`remix.ethereum.org`), loaded the contract, and successfully compiled it.

### 3. On-Chain Contract Deployment

- In Remix, set the deployment environment to "**Injected Provider - MetaMask**" (Sepolia network).
- Deployed the contract to the blockchain, signed the transaction in MetaMask, and recorded the resulting **Contract Address** (`0x43aFd...`) and **ABI**.

### 4. Frontend Development & Web3 Integration

- Developed the user interface (`index.html`) matching the required sketch (Deposit, Withdraw, Transfer, Check Balance, and Account List).
- Imported **ethers.js (v6)** to bridge the HTML interface with the MetaMask provider.
- Implemented automatic network switching (`wallet_switchEthereumChain`) to programmatically lock the app to Sepolia.
- Configured the JavaScript to bind button clicks to contract functions using the Contract Address and ABI.

### 5. Deployment & Multi-User Testing

- Served the frontend locally via **VS Code Live Server** (`http://127.0.0.1:5500`).

- Connected **Account 1**: Registered the account, verified receipt of 100 tokens, deposited 0.01 ETH, and executed a partial withdrawal.
- Added **Account 2** in MetaMask: Funded it with gas ETH, registered it, and performed peer-to-peer token transfers between both accounts to verify state updates on the blockchain.

## 1.Contract Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.20;
```

```
contract SimpleBank {
```

```
    // Total supply of 1,000,000 tokens and 100 token bonus per account
```

```
    uint256 public constant INITIAL_BALANCE = 100;
```

```
    uint256 public remainingSupply = 1000000;
```

```
    mapping(address => uint256) private ethBalances;
```

```
    mapping(address => uint256) private tokenBalances;
```

```
    mapping(address => bool) public registered;
```

```
    address[] public accounts;
```

```
    event AccountRegistered(address indexed user, uint256 tokenBonus);
```

```
    event Deposited(address indexed user, uint256 amount);
```

```
    event Withdrawn(address indexed user, uint256 amount);
```

```
    event Transferred(address indexed from, address indexed to, uint256 amount);
```

```
    modifier onlyRegistered() {
```

```
        require(registered[msg.sender], "Account not registered");
```

```
        _;
```

```
    }
```

```
// 1. registernewaccount(): Registers user & allocates 100 tokens from 1,000,000 supply
```

```
function registernewaccount() external {
```

```
    require(!registered[msg.sender], "Account already registered");
```

```
    require(remainingSupply >= INITIAL_BALANCE, "Supply exhausted");
```

```
    registered[msg.sender] = true;
    tokenBalances[msg.sender] = INITIAL_BALANCE;
    remainingSupply -= INITIAL_BALANCE;
    accounts.push(msg.sender);

    emit AccountRegistered(msg.sender, INITIAL_BALANCE);
}

// 2. deposit() -> payable: Accepts ETH and credits user balance
function deposit() external payable onlyRegistered {
    require(msg.value > 0, "Deposit must be greater than 0");
    ethBalances[msg.sender] += msg.value;
    emit Deposited(msg.sender, msg.value);
}

// 3. withdraw(uint amount) -> withdraws deposited ETH
function withdraw(uint256 amount) external onlyRegistered {
    require(amount > 0, "Amount must be greater than 0");
    require(ethBalances[msg.sender] >= amount, "Insufficient ETH balance");

    ethBalances[msg.sender] -= amount;
    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "ETH transfer failed");

    emit Withdrawn(msg.sender, amount);
}

// 4. transfer(address to, uint amount) -> transfers tokens between users
function transfer(address to, uint256 amount) external onlyRegistered {
    require(to != address(0), "Invalid recipient");
    require(registered[to], "Recipient not registered");
    require(amount > 0, "Amount must be greater than 0");
```

```
require(tokenBalances[msg.sender] >= amount, "Insufficient token balance");

tokenBalances[msg.sender] -= amount;
tokenBalances[to] += amount;

emit Transferred(msg.sender, to, amount);
}

// 5. balanceOf(address user) -> returns token balance
function balanceOf(address user) external view returns (uint256) {
    return tokenBalances[user];
}

// Helper view functions
function getEthBalance(address user) external view returns (uint256) {
    return ethBalances[user];
}

function getAccounts() external view returns (address[] memory) {
    return accounts;
}

function contractBalance() external view returns (uint256) {
    return address(this).balance;
}
}
```

## 2.Frontend Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Simple Banking App</title>
```

```

<script src="https://cdn.jsdelivr.net/npm/ethers@6.13.4/dist/ethers.umd.min.js"></script>
<style>
  body { font-family: Arial, sans-serif; background: #f4f6f8; margin: 0; padding: 25px; }
  .container { max-width: 800px; margin: auto; }
  h1, .wallet { text-align: center; margin-bottom: 20px; }
  .card { background: white; padding: 20px; margin-bottom: 15px; border-radius: 10px;
box-shadow: 0 2px 8px rgba(0,0,0,0.06); }
  input { width: 100%; padding: 10px; margin: 6px 0; border: 1px solid #ccc; border-
radius: 5px; box-sizing: border-box; }
  button { padding: 10px 18px; margin-top: 6px; border: none; border-radius: 5px; cursor:
pointer; background: #2563eb; color: white; font-weight: bold; }
  #connectBtn { background: #111827; }
  .dashboard-grid { display: flex; gap: 10px; margin-top: 10px; }
  .stat-card { flex: 1; padding: 12px; background: #eff6ff; border-radius: 6px; text-align:
center; border: 1px solid #bfdbfe; }
  .stat-card strong { display: block; font-size: 22px; color: #1e3a8a; }
  .account-badge { background: #f3f4f6; padding: 8px; border-radius: 5px; margin-top:
5px; word-break: break-all; }
  #status { text-align: center; font-weight: bold; margin: 15px 0; color: #d97706; }
</style>
</head>
<body>
  <div class="container">
    <h1>  Simple Banking App</h1>

    <!-- Connect MetaMask -->
    <div class="wallet">
      <button id="connectBtn">Connect MetaMask</button>
      <p><strong>Connected Account:</strong> <span id="userAddress">Not
Connected</span></p>
    </div>

    <!-- 1. Check Balance Dashboard -->
    <div class="card">
      <h2>Account Dashboard</h2>
      <button onclick="checkBalance()">Check Balance</button>
      <div class="dashboard-grid">
        <div class="stat-card">Bank Tokens (Bonus)<strong
id="tokenBalance">0</strong></div>
        <div class="stat-card">Deposited ETH<strong id="ethBalance">0
ETH</strong></div>
      </div>
    </div>

    <!-- 2. Register -->
    <div class="card">
      <h2>Register New Account</h2>
      <p>New accounts receive 100 BANK tokens from the 1,000,000 supply pool.</p>
      <button onclick="registerAccount()">Register Account</button>
    </div>
  </div>

```

```

<!-- 3. Deposit Button with ETH amount -->
<div class="card">
  <h2>Deposit (ETH)</h2>
  <input type="number" id="depositAmount" placeholder="Enter ETH amount (e.g.
0.01)" step="0.0001">
  <button onclick="deposit()">Deposit</button>
</div>

<!-- 4. Withdraw Button -->
<div class="card">
  <h2>Withdraw (ETH)</h2>
  <input type="number" id="withdrawAmount" placeholder="Enter ETH amount to
withdraw" step="0.0001">
  <button onclick="withdraw()">Withdraw</button>
</div>

<!-- 5. Transfer input + button -->
<div class="card">
  <h2>Transfer (Tokens)</h2>
  <input type="text" id="recipient" placeholder="Recipient wallet address (0x...)">
  <input type="number" id="transferAmount" placeholder="Enter number of tokens"
step="1">
  <button onclick="transfer()">Transfer</button>
</div>

<!-- 6. List all accounts -->
<div class="card">
  <h2>List All Accounts</h2>
  <button onclick="loadAccounts()">List Accounts</button>
  <div id="accountsList" style="margin-top: 10px;"></div>
</div>

<div id="status">Ready.</div>
</div>

<script>
const CONTRACT_ADDRESS = "Enter Your Contract Address here";
const CONTRACT_ABI = [
  "function registernewaccount() external",
  "function deposit() external payable",
  "function withdraw(uint256 amount) external",
  "function transfer(address to, uint256 amount) external",
  "function balanceOf(address user) external view returns (uint256)",
  "function getEthBalance(address user) external view returns (uint256)",
  "function getAccounts() external view returns (address[])",
  "function contractBalance() external view returns (uint256)",
  "function registered(address) external view returns (bool)"
];

let provider, signer, contract;
document.getElementById("connectBtn").onclick = connectWallet;

```

```
async function connectWallet() {
  if (!window.ethereum) return alert("MetaMask not found!");
  try {
    await window.ethereum.request({
      method: 'wallet_switchEthereumChain',
      params: [{ chainId: '0xaa36a7' }] // Sepolia
    });
  } catch (e) { console.log(e); }

  provider = new ethers.BrowserProvider(window.ethereum);
  await provider.send("eth_requestAccounts", []);
  signer = await provider.getSigner();
  contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);

  const addr = await signer.getAddress();
  document.getElementById("userAddress").innerText = addr;
  showStatus("Wallet connected.");
  await checkBalance();
}

async function registerAccount() {
  if (!contract) return showStatus("Connect MetaMask first.");
  try {
    showStatus("Registering...");
    const tx = await contract.registerNewAccount();
    await tx.wait();
    showStatus("Registered! 100 Tokens received.");
    await checkBalance();
    await loadAccounts();
  } catch (err) { showStatus(getErrorMessage(err)); }
}

async function deposit() {
  if (!contract) return showStatus("Connect MetaMask first.");
  const val = document.getElementById("depositAmount").value;
  if (!val || Number(val) <= 0) return alert("Enter valid ETH.");
  try {
    showStatus("Depositing ETH...");
    const tx = await contract.deposit({ value: ethers.parseEther(val) });
    await tx.wait();
    showStatus("Deposit successful.");
    document.getElementById("depositAmount").value = "";
    await checkBalance();
  } catch (err) { showStatus(getErrorMessage(err)); }
}

async function withdraw() {
  if (!contract) return showStatus("Connect MetaMask first.");
  const val = document.getElementById("withdrawAmount").value;
  if (!val || Number(val) <= 0) return alert("Enter valid ETH.");
```

```

    try {
      showStatus("Withdrawing ETH...");
      const tx = await contract.withdraw(ethers.parseEther(val));
      await tx.wait();
      showStatus("Withdrawal successful.");
      document.getElementById("withdrawAmount").value = "";
      await checkBalance();
    } catch (err) { showStatus(getErrorMessage(err)); }
  }

  async function transfer() {
    if (!contract) return showStatus("Connect MetaMask first.");
    const to = document.getElementById("recipient").value;
    const amt = document.getElementById("transferAmount").value;
    if (!ethers.isAddress(to) || !amt || Number(amt) <= 0) return alert("Invalid inputs.");
    try {
      showStatus("Transferring tokens...");
      const tx = await contract.transfer(to, amt);
      await tx.wait();
      showStatus("Transfer successful.");
      document.getElementById("recipient").value = "";
      document.getElementById("transferAmount").value = "";
      await checkBalance();
    } catch (err) { showStatus(getErrorMessage(err)); }
  }

  async function checkBalance() {
    if (!contract || !signer) return;
    const addr = await signer.getAddress();
    const tokens = await contract.balanceOf(addr);
    const eth = await contract.getEthBalance(addr);
    document.getElementById("tokenBalance").innerText = tokens.toString();
    document.getElementById("ethBalance").innerText = ethers.formatEther(eth) + "
ETH";
  }

  async function loadAccounts() {
    if (!contract) return;
    const accounts = await contract.getAccounts();
    const list = document.getElementById("accountsList");
    list.innerHTML = accounts.map((a, i) => `<div class="account-
badge"><strong>Account ${i+1}</strong> ${a}</div>`).join("");
  }

  function getErrorMessage(e) {
    return e.reason || e.shortMessage || (e.message.includes("user rejected") ? "Rejected by
user." : "Transaction failed.");
  }

  function showStatus(msg) { document.getElementById("status").innerText = msg; }

```

```

if (window.ethereum) {
  window.ethereum.on("accountsChanged", async () => {
    if (!provider) return;
    signer = await provider.getSigner();
    contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI,
signer);
    document.getElementById("userAddress").innerText = await signer.getAddress();
    await checkBalance();
  });
}
</script>
</body>
</html>

```

### 3. Test Cases:

| TC ID | Test Case                          | Preconditions                                    | Test Steps                                                                                    | Expected Result                                               | Status |
|-------|------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------|--------|
| TC01  | Wallet Connection & Network Switch | MetaMask installed;<br>DApp open on local server | 1. Click "Connect MetaMask"<br>2. Confirm connection in popup                                 | Wallet connects;<br>UI switches to Sepolia; address displayed | PASS   |
| TC02  | New Account Registration           | Wallet connected;<br>account not registered      | 1. Click "Register Account"<br>2. Confirm transaction in MetaMask<br>3. Click "Check Balance" | Transaction mined; user receives 100 Bank Tokens              | PASS   |
| TC03  | Duplicate Registration Prevention  | Account already registered                       | 1. Click "Register Account" again                                                             | Transaction reverts with "Account already registered"         | PASS   |
| TC04  | Deposit Native ETH                 | Account registered;<br>wallet holds Sepolia ETH  | 1. Enter 0.01 in Deposit field<br>2. Click "Deposit"<br>3. Confirm in MetaMask                | 0.01 ETH transferred to contract;<br>dashboard shows 0.01 ETH | PASS   |
| TC05  | Withdraw Native ETH                | Account has deposited ETH in contract            | 1. Enter 0.01 in Withdraw field<br>2. Click "Withdraw"<br>3. Confirm in MetaMask              | 0.01 ETH returned to wallet; dashboard shows 0.0 ETH          | PASS   |
| TC06  | Over-withdrawal Prevention         | Deposited ETH balance is 0.0 ETH                 | 1. Enter 1.0 in Withdraw field                                                                | Transaction reverts                                           | PASS   |

|             |                                   |                                                       |                                                                           |                                                                             |             |
|-------------|-----------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------|-------------|
|             |                                   |                                                       | 2. Click "Withdraw"                                                       | with "Insufficient ETH balance"                                             |             |
| <b>TC07</b> | Peer-to-Peer Token Transfer       | Both accounts registered; Sender has $\geq 10$ tokens | 1. Enter Recipient address<br>2. Enter 10 tokens<br>3. Click "Transfer"   | Sender tokens: 100 $\rightarrow$ 90; Receiver tokens: 100 $\rightarrow$ 110 | <b>PASS</b> |
| <b>TC08</b> | Transfer to Unregistered Account  | Sender registered; Recipient not registered           | 1. Enter unregistered address<br>2. Enter 5 tokens<br>3. Click "Transfer" | Transaction reverts with "Recipient not registered"                         | <b>PASS</b> |
| <b>TC09</b> | Check Account Balance (balanceOf) | Wallet connected                                      | 1. Click "Check Balance" button                                           | Displays current tokens and deposited ETH with 0 gas cost                   | <b>PASS</b> |
| <b>TC10</b> | List All Registered Accounts      | At least one account registered                       | 1. Click "List Accounts" button                                           | Displays all registered user addresses dynamically on screen                | <b>PASS</b> |

## Output:

### TC01:

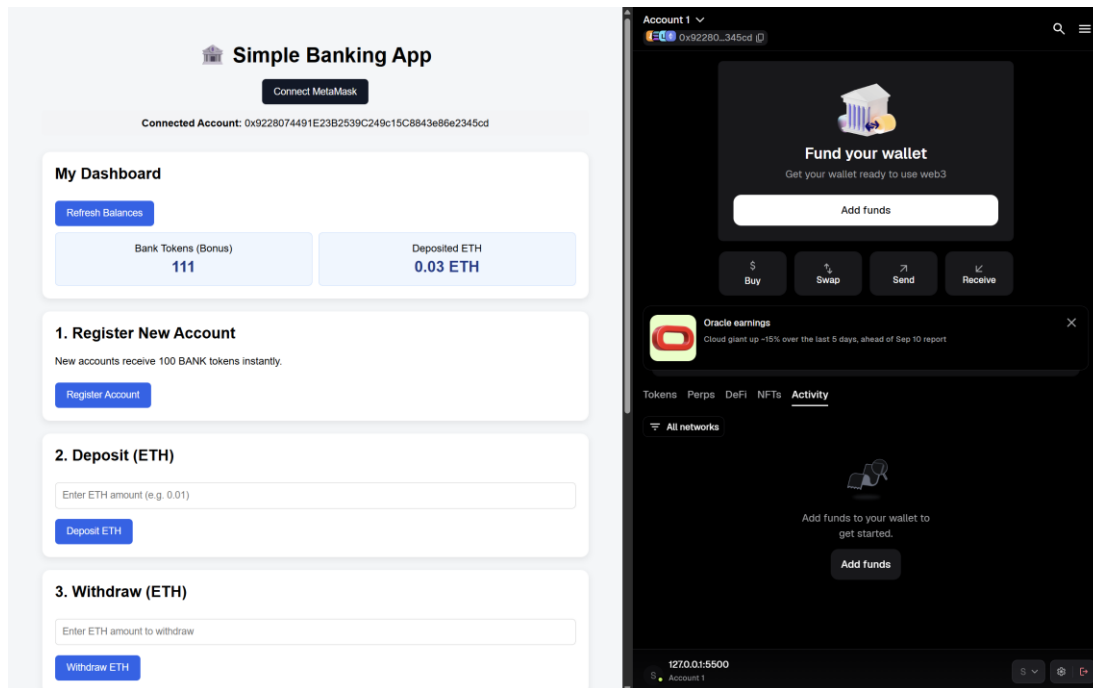


Figure 1: Successful connection of MetaMask wallet to the DApp on Sepolia Testnet.

### TC02:

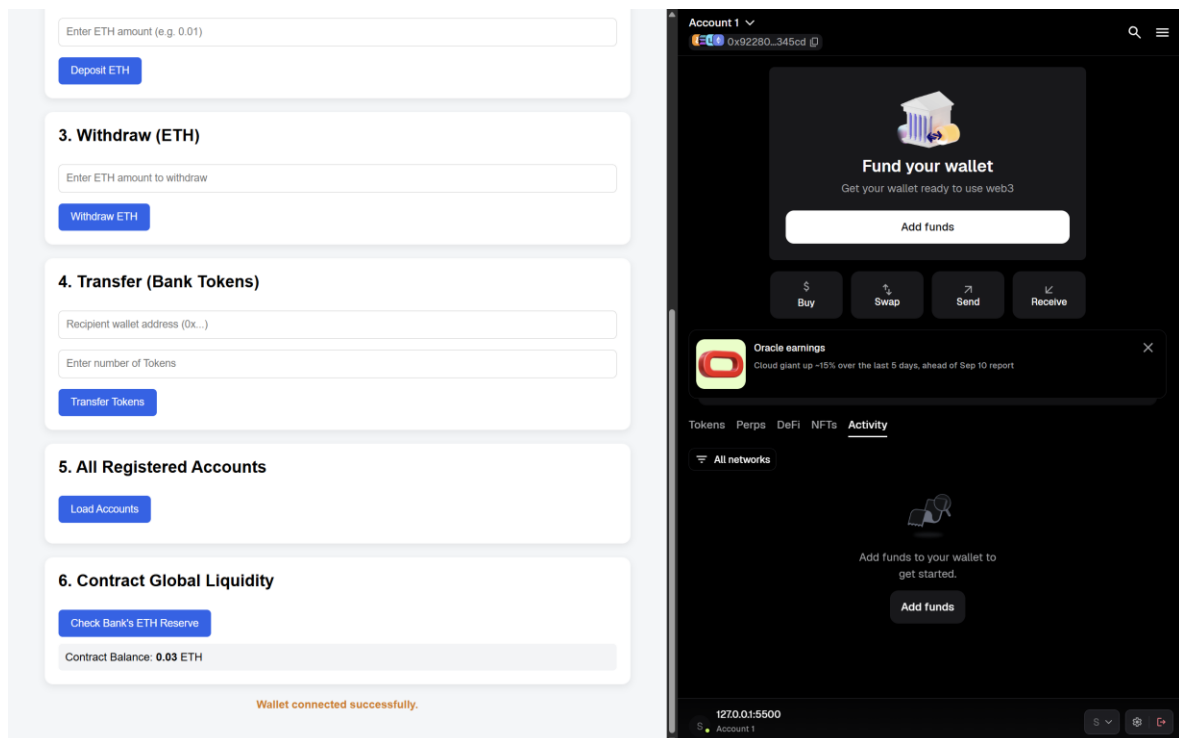


Figure 2: New account registration confirming receipt of initial 100 Bank Tokens.

TC03:

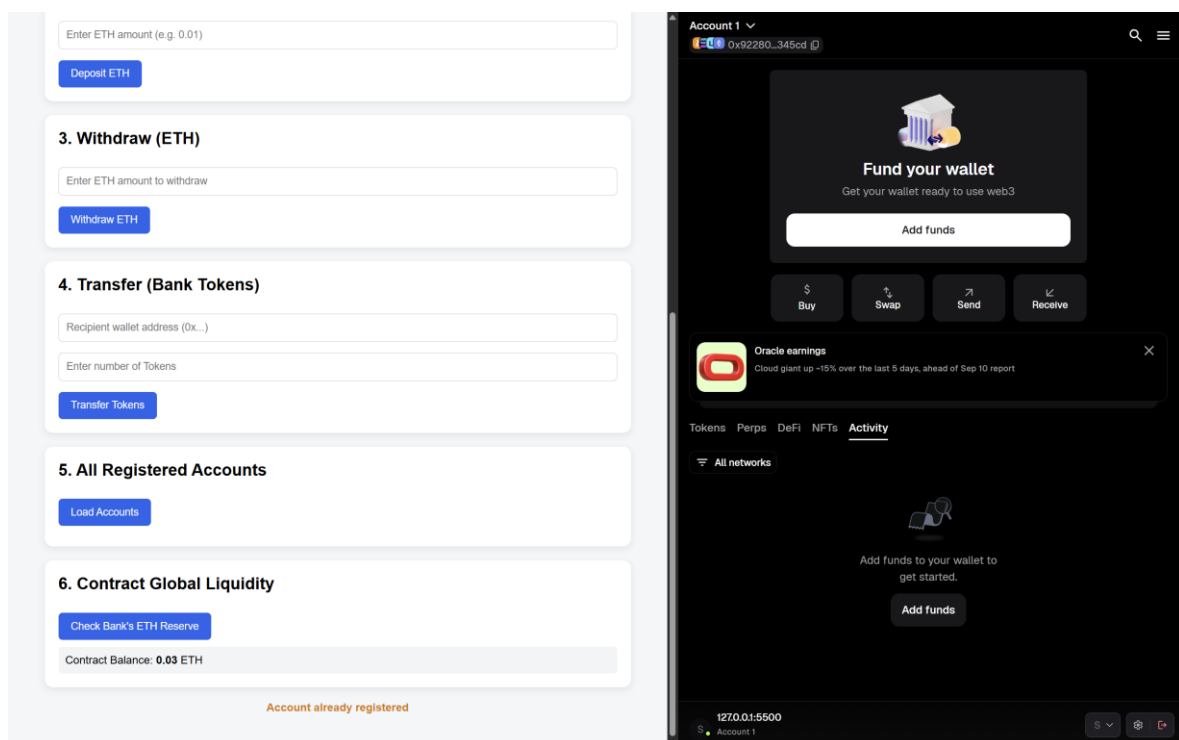


Figure 3: Smart contract requirement preventing already registered user from registering again.

TC04:

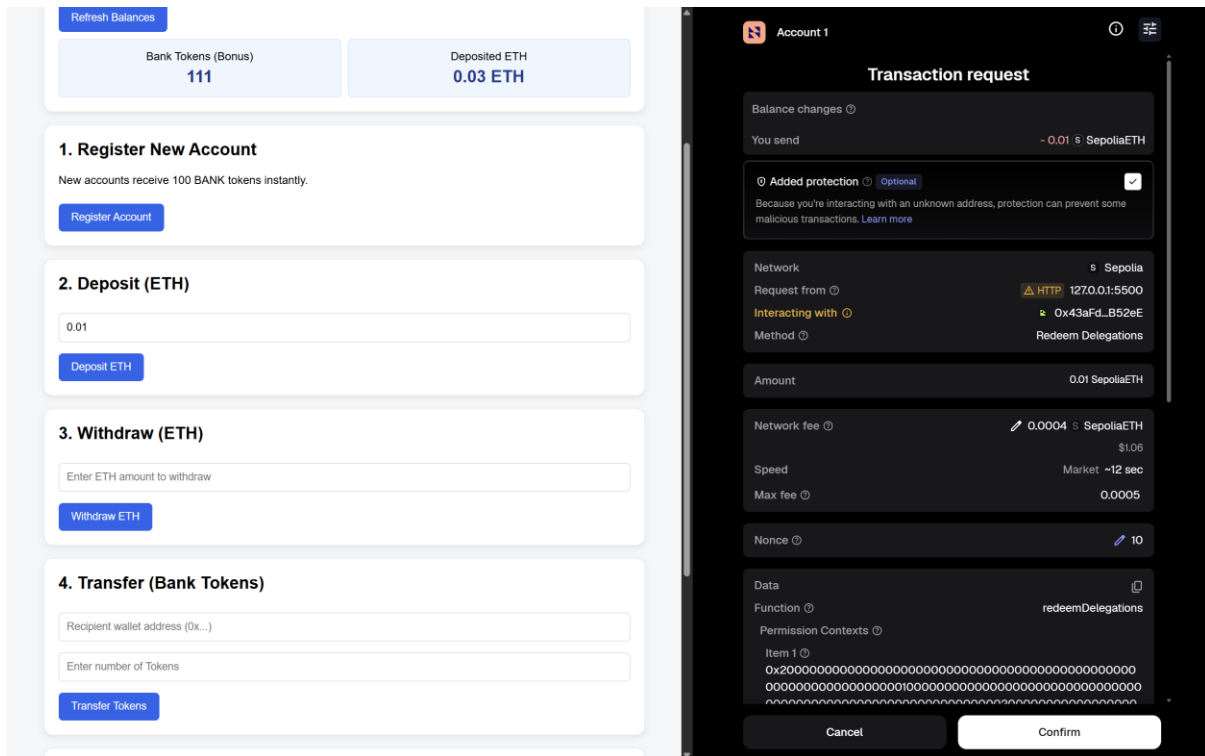


Figure 4: Depositing 0.01 Sepolia ETH into the smart contract vault via payable function.

TC05:

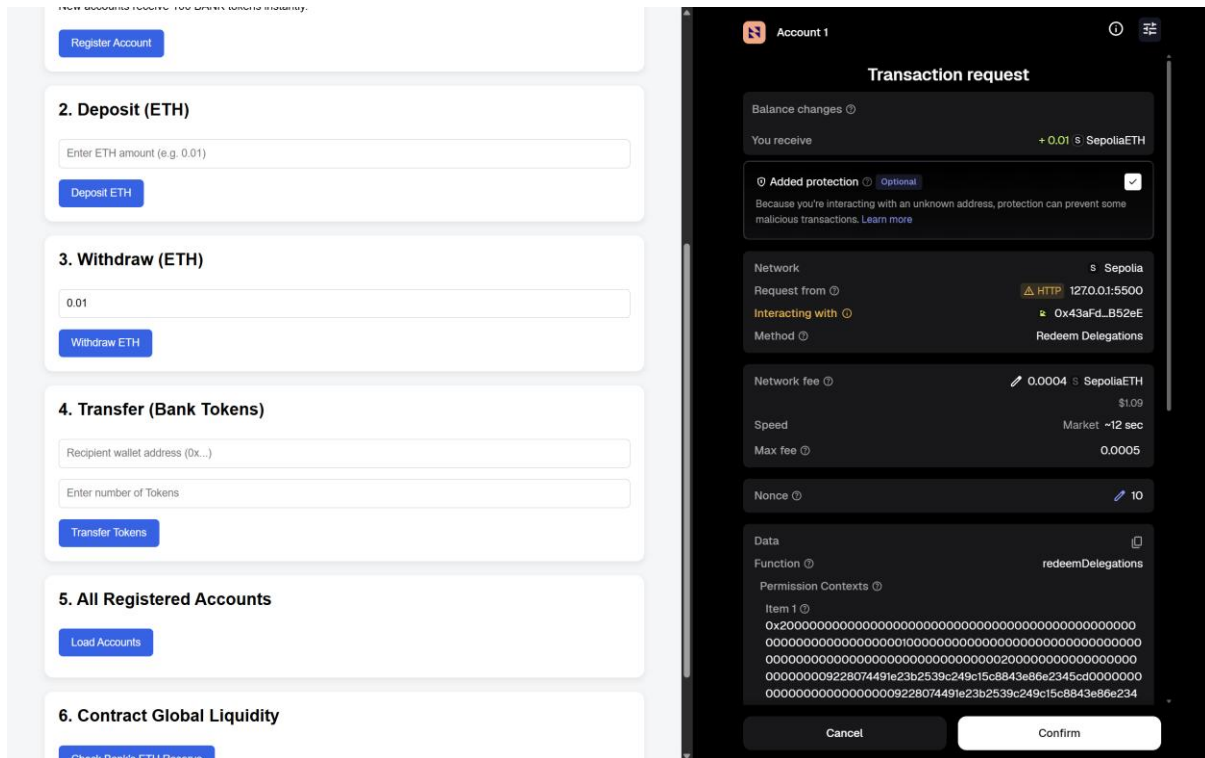


Figure 5: Withdrawing 0.01 ETH back to user's wallet with balance update.

TC06:

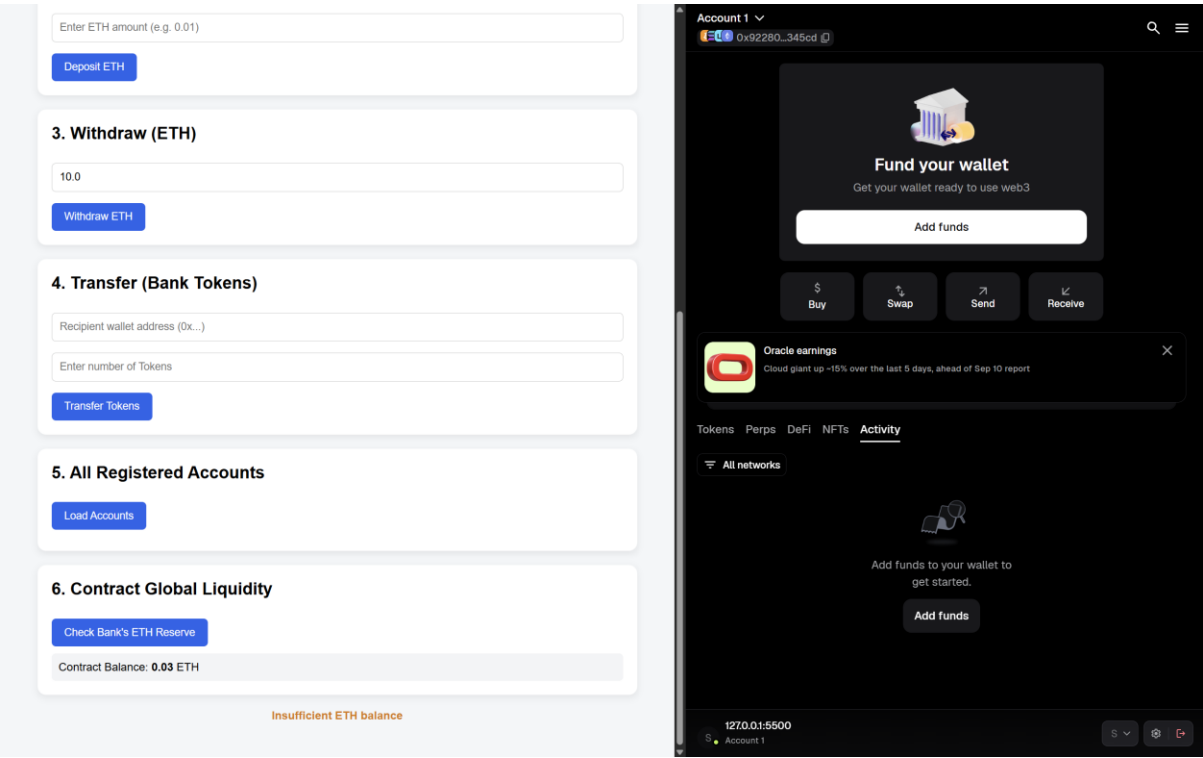


Figure 6: Smart contract revert verifying that users cannot withdraw more ETH than deposited.

TC07:

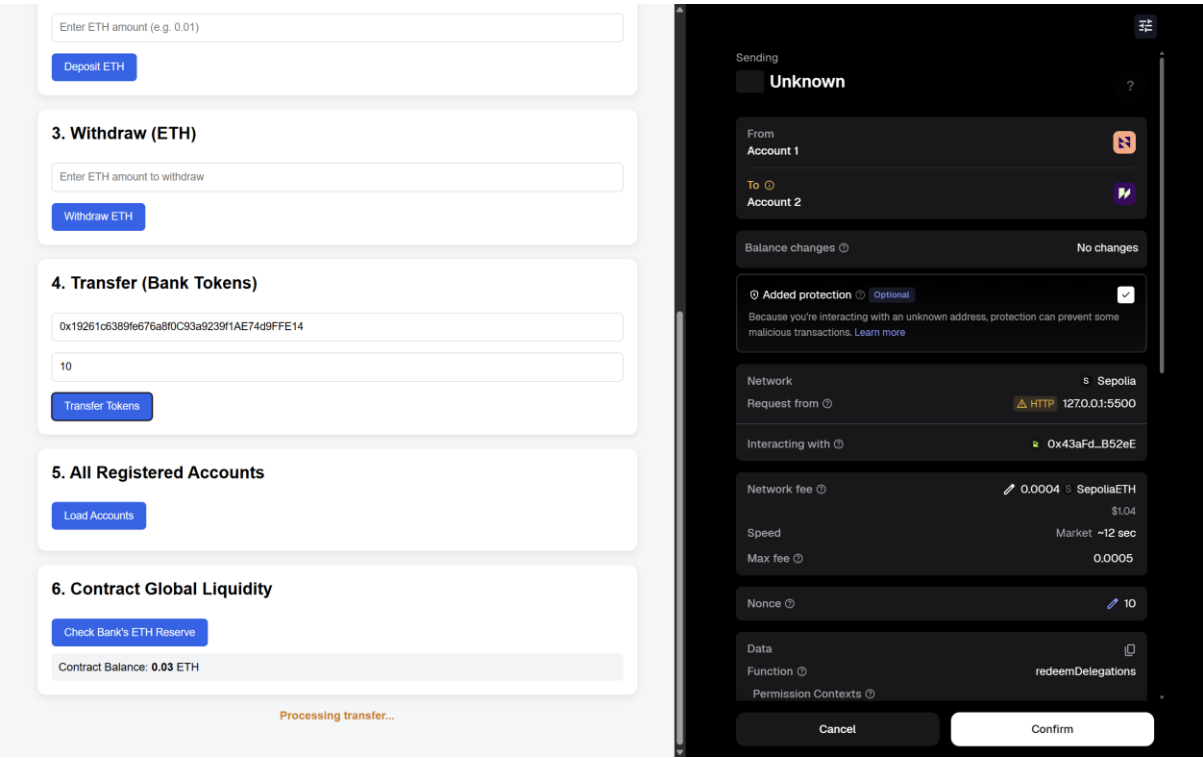


Figure 7: Successful on-chain transfer of tokens from Account 2 to Account 1.

TC08:

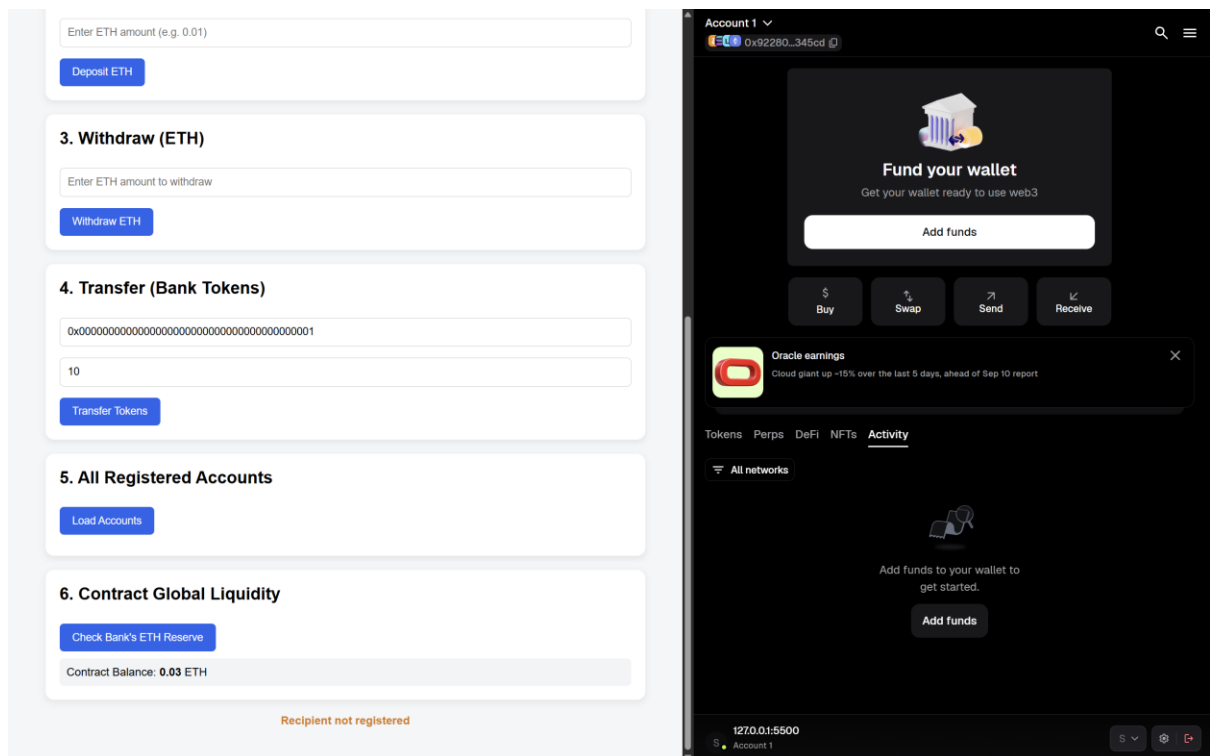


Figure 8: Security validation preventing transfer to non-whitelisted/unregistered addresses.

TC09:

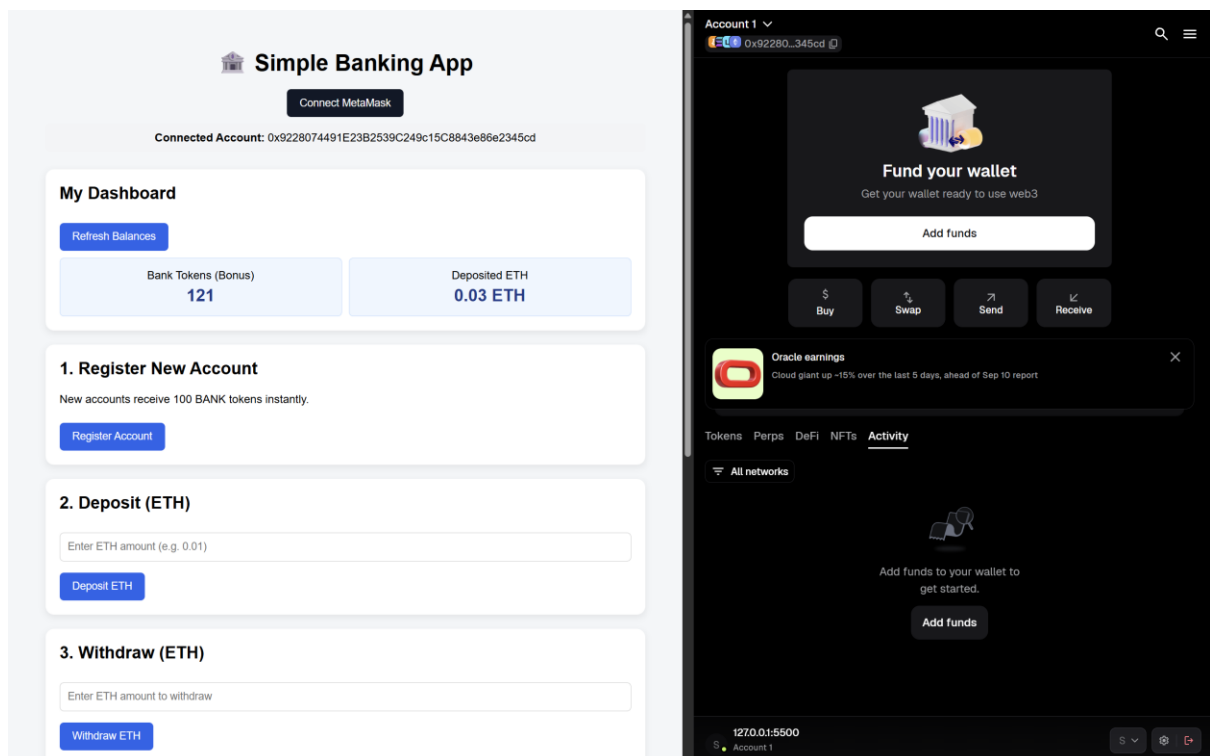


Figure 9: Balance query demonstrating Account 1 balance increased to 121 tokens after receiving transfer.

## TC10:

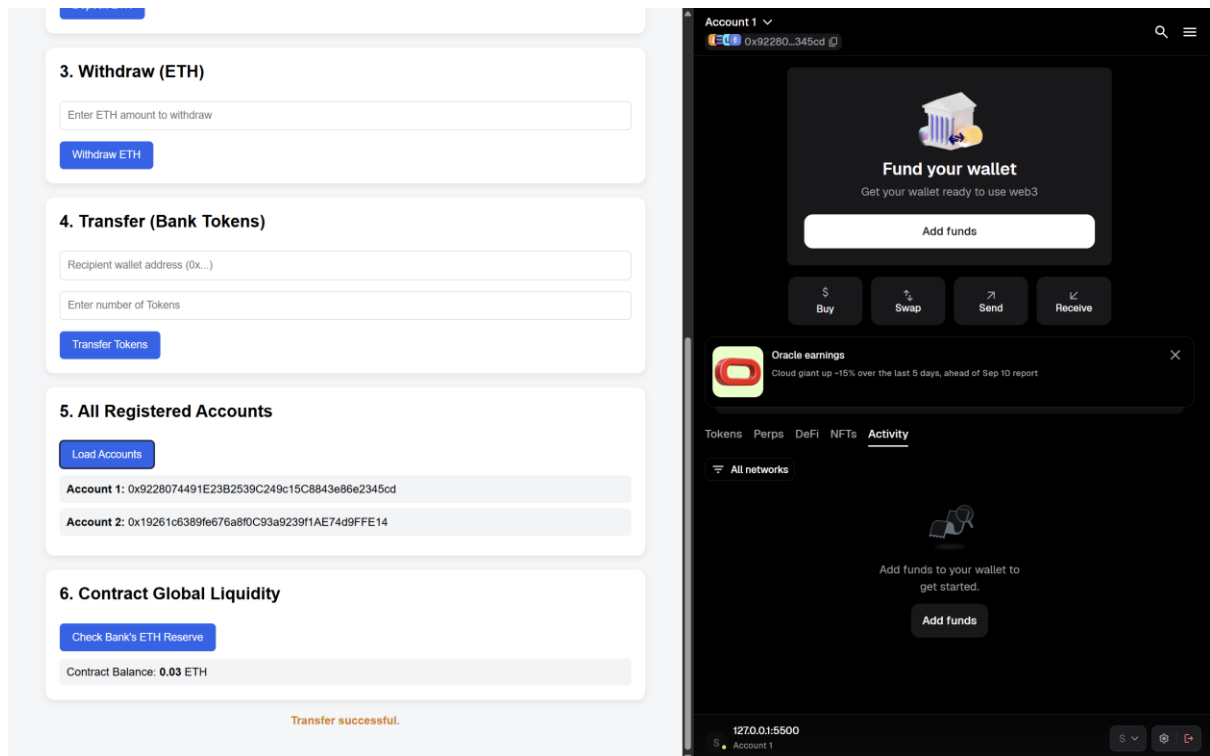


Figure 10: Dynamic list of all registered accounts and total contract liquidity reserve.